

Method Breakdown: initializeSelect2()	3
1. Function Declaration	3
2. Debug Logging	3
3. Safety Check - Library Availability	3
4. Error Handling	3
5. Main Initialization Block	4
6. Configuration Options	4
Placeholder Text	4
Clear Button	4
Width Setting	5
Language Configuration	5
How This Method Works in Practice:	5
Key Things to Understand:	5
Method Breakdown: jQuery Document Ready	7
1. jQuery Object Access	7
2. The Parameter: function (\$)	7
3. Function Body	7
4. Closing Brackets	8
What This Pattern Accomplishes:	8
DOM Ready vs Window Load	8
Execution Timeline:	8
Alternative Ways to Write This:	8
1. Using \$ Shorthand (if safe)	8
2. Shorter Syntax	9
3. Modern Alternative (ES6+)	9
4. Arrow Function (ES6+)	9
Why This Pattern Exists:	9
Problem Without Document Ready:	9
Solution With Document Ready:	9
In Your Context:	10
Common Use Cases:	10
Method-by-Method Analysis of Your CustomSelectEditor	11

1. CustomSelectEditor() - Constructor.....	11
2. init(params) - Initialization Method.....	11
3. getGui() - Returns DOM Element	13
4. afterGuiAttached() - Post-Attachment Logic	13
5. getValue() - Returns Selected Value	14
6. destroy() - Cleanup.....	14
7. isPopup() - Popup Indicator	14
8. onKeyDown() - Keyboard Handling.....	14
Method-by-Method Analysis of Your CustomTextFilter	16
1. Constructor	16
2. init(params) - The Main Setup Method	16
2A. Initial Setup	16
2B. Main Container Creation.....	16
2C. Filter Header Section (Dropdown)	17
2D. Filter Type Dropdown	17
2E. Dropdown Options	18
2F. Search Input Section	19
2G. Text Input Field	19
2H. Clear Button Section	20
2I. Event Listeners.....	20
2J. State Restoration	21
3. afterGuiAttached() - Post-Display Setup.....	21
4. getGui() - Returns Filter UI	22
5. isFilterActive() - Filter State Check	22
6. doesFilterPass(params) - The Core Filtering Logic.....	1
7. getModel() - Save Filter State	25
8. setModel(model) - Restore Filter State	25
9. Event Handlers.....	26
9A. onFilterTypeChanged()	26
9B. onFilterChanged()	26
9C. onClearFilter()	27
10. getModelAsString() - Display Current Filter	27

Method Breakdown: initializeSelect2()

1. Function Declaration

```
javascript
function initializeSelect2() {
```

- **What it is:** A regular JavaScript function declaration
- **Purpose:** Creates a reusable function to initialize Select2 on elements
- **Scope:** Can be called from anywhere in the code after it's defined

2. Debug Logging

```
javascript
console.log('Inicializando Select2');
```

- **What it does:** Prints "Inicializando Select2" (Initializing Select2) to browser console
- **Purpose:** Helps developers track when this function runs
- **When you'll see it:** Open browser DevTools → Console tab when this function executes

3. Safety Check - Library Availability

```
javascript
if ($.fn.select2 === undefined) {
```

Breaking this down:

- `$` = jQuery object (alias for jQuery)
- `$.fn` = jQuery's prototype (where all jQuery methods live)
- `$.fn.select2` = The Select2 plugin method added to jQuery
- `=== undefined` = Strict equality check to see if Select2 plugin exists
- **Translation:** "If the Select2 plugin is not loaded/available..."

4. Error Handling

```
javascript
```

```
console.error('Select2 não está disponível');
```

- **What it does:** Prints red error message "Select2 não está disponível" (Select2 is not available)
- **When it runs:** Only if Select2 library isn't loaded
- **Purpose:** Prevents JavaScript errors and alerts developer to missing dependency

5. Main Initialization Block

```
javascript
} else {
    $('.select2').select2({
        // configuration options
    });
}
```

Breaking this down:

- `else` = "If Select2 IS available, then do this..."
- `$(' .select2 ')` = jQuery selector that finds ALL elements with CSS class "select2"
- `.select2({ ... })` = Calls the Select2 initialization method with configuration options

6. Configuration Options

Placeholder Text

```
javascript
placeholder: "Selecione uma opção",
```

- **What it does:** Sets the text shown when no option is selected
- **Translation:** "Select an option"

Clear Button

```
javascript
allowClear: true,
```

- **What it does:** Adds an "X" button to clear the selection
- **Effect:** Users can click "X" to deselect and return to placeholder state

Width Setting

```
javascript
width: '100%',
```

- **What it does:** Makes Select2 take up 100% of its container's width
- **Alternative options:** '200px', 'auto', 'resolve', etc.

Language Configuration

```
javascript
language: {
  noResults: function () {
    return "Nenhum resultado encontrado";
  }
}
```

Breaking this down:

- `language: {...}` = Localization settings object
- `noResults: function() {...}` = Callback function for "no results" message
- **Purpose:** When user searches and no matches found, shows "Nenhum resultado encontrado" (No results found)
- **Note:** This is a function that returns a string (allows for dynamic messages)

How This Method Works in Practice:

1. **Function gets called** → `initializeSelect2()`
2. **Logs start message** → Console shows "Iniciando Select2"
3. **Checks if Select2 exists** → Prevents errors if library not loaded
4. **If Select2 missing** → Shows error, stops execution
5. **If Select2 available** → Continues to initialization
6. **Finds all elements** → Looks for `<select class="select2">` elements
7. **Initializes each one** → Converts them to fancy Select2 dropdowns
8. **Applies configuration** → Sets placeholder, clear button, width, language

Key Things to Understand:

- **Global Selector:** `$('.select2')` affects ALL elements with class "select2" on the page
- **Batch Initialization:** This initializes multiple Select2 instances at once

- **Safety First:** Checks library availability before using it
- **Portuguese Localization:** Custom messages in Portuguese
- **Defensive Programming:** Won't crash if Select2 isn't loaded

This is a **utility function** designed to be called when the page loads or when new Select2 elements are added to the DOM.

Method Breakdown: jQuery Document Ready

1. jQuery Object Access

```
javascript
jQuery(document).ready(function ($) {
```

Breaking this down:

- **jQuery** = The full jQuery library object (not the \$ shorthand)
- **(document)** = Passes the HTML document object to jQuery
- **.ready()** = jQuery method that waits for DOM to be fully loaded
- **function (\$) {** = Anonymous callback function that receives jQuery as parameter \$

Why use jQuery instead of \$?

- **Conflict Prevention:** Other libraries might use \$ symbol
- **Explicit:** Makes it clear we're using jQuery
- **Best Practice:** Ensures jQuery is available even if \$ is overridden

2. The Parameter: function (\$)

```
javascript
function ($) {
```

What this does:

- **Parameter Mapping:** Maps the full jQuery object to the shorthand \$
- **Local Scope:** Inside this function, \$ definitely refers to jQuery
- **Safety:** Even if \$ is used by another library globally, inside this function it's jQuery
- **Alternative:** You could write function (jq) and use jq instead of \$

3. Function Body

```
javascript
    initializeSelect2();
```

- **Simple Call:** Executes the initializeSelect2() function we analyzed earlier
- **Context:** Runs after DOM is ready but before images/external resources load

4. Closing Brackets

```
javascript
});
```

- First `}` closes the anonymous function
- `)` closes the `.ready()` method call
- `;` ends the statement

What This Pattern Accomplishes:

DOM Ready vs Window Load

```
javascript
// DOM Ready (what this code uses)
jQuery(document).ready(function ($) {
    // Runs when HTML structure is loaded
    // Images, stylesheets may still be loading
});

// vs Window Load
jQuery(window).load(function ($) {
    // Runs when EVERYTHING is loaded (images, etc.)
    // Takes longer to execute
});
```

Execution Timeline:

1. **Browser starts loading page**
2. **HTML structure loads** ← DOM Ready fires here
3. **CSS, images, external resources load**
4. **Window load event fires**

Alternative Ways to Write This:

1. Using \$ Shorthand (if safe)

```
javascript
```



```
$(document).ready(function() {  
    initializeSelect2();  
});
```

2. Shorter Syntax

```
javascript  
jQuery(function($) {  
    initializeSelect2();  
});
```

3. Modern Alternative (ES6+)

```
javascript  
document.addEventListener('DOMContentLoaded', function() {  
    initializeSelect2();  
});
```

4. Arrow Function (ES6+)

```
javascript  
jQuery(document).ready(() => {  
    initializeSelect2();  
});
```

Why This Pattern Exists:

Problem Without Document Ready:

```
javascript  
// BAD - This might run before HTML elements exist  
initializeSelect2(); // Could fail if <select> elements not  
loaded yet
```

Solution With Document Ready:

```
javascript  
// GOOD - This waits for HTML elements to exist  
jQuery(document).ready(function ($) {
```

```
    initializeSelect2(); // Will work because DOM is ready
});
```

In Your Context:

This code ensures that:

1. **DOM Elements Exist:** All `<select class="select2">` elements are loaded
2. **jQuery is Available:** The `$` parameter guarantees jQuery access
3. **Select2 Can Initialize:** Elements exist for Select2 to enhance
4. **No Race Conditions:** Won't try to initialize before elements are ready

Common Use Cases:

```
javascript
jQuery(document).ready(function ($) {
    // Initialize plugins
    initializeSelect2();

    // Bind events
    $('.button').click(function() { ... });

    // Set up components
    $('.datepicker').datepicker();

    // Any DOM manipulation that needs elements to exist
});
```

Bottom Line: This is jQuery's way of saying "Wait until the page structure is ready, then run my initialization code safely."

Method-by-Method Analysis of Your CustomSelectEditor

1. CustomSelectEditor() - Constructor

javascript

```
function CustomSelectEditor() {}
```

- **Purpose:** Empty constructor function
- **Issues:** No initialization logic here (which is fine, AG Grid calls `init()` separately)

2. init(params) - Initialization Method

javascript

```
CustomSelectEditor.prototype.init = function(params) {  
    this.params = params;  
    this.isDestroyed = false;  
    // ... rest of method  
}
```

What it does step by step:

1. **Store parameters:** `this.params = params` - Stores AG Grid's parameters
2. **Create wrapper div:**

javascript

```
this.eWrapper = document.createElement('div');  
this.eWrapper.style.width = '200px';  
this.eWrapper.style.height = '100%';
```

- a. Creates a container div with fixed 200px width

3. **Create select element:**

javascript

```
this.eInput = document.createElement('select');  
this.eInput.style.width = '100%';  
this.eInput.style.height = '100%';
```

- a. Creates the actual `<select>` element

4. **Determine data source:**

javascript

```
let idToNameMap;  
if (params.column.colDef.field === 'motoristaId') {
```

```

    idToNameMap = params.context.professionaisDict;
  } // ... etc

```

a. Chooses which dictionary to use based on the column field name

5. Add empty option:

javascript

```

const emptyOption = document.createElement('option');
emptyOption.value = '';
emptyOption.text = 'Selecione...';
this.eInput.appendChild(emptyOption);

```

a. Adds a placeholder option

6. Populate options:

javascript

```

if (params.values && params.values.length > 0) {
  params.values.forEach(value => {
    const option = document.createElement('option');
    option.value = value;
    option.text = idToNameMap[value] || value;
    this.eInput.appendChild(option);
  });
}

```

a. Loops through available values and creates option elements

7. Set current value:

javascript

```

if (params.value) {
  this.eInput.value = params.value;
}

```

a. Pre-selects the current cell value

8. Add select to wrapper: this.eWrapper.appendChild(this.eInput);

9. Initialize Select2 (inside setTimeout):

javascript

```

setTimeout(() => {
  $(this.eInput).select2({
    dropdownParent: document.body, // 🚩 PROBLEM #1
    width: '100%',
    placeholder: 'Selecione...',
    allowClear: true
  });
});

```

a. **ISSUE:** dropdownParent: document.body causes focus problems

10. Event handling:

javascript

```
const select2Container = $(this.eInput).next('.select2-container');
select2Container.on('click mousedown touchend', (e) => {
    e.stopPropagation(); // 🚩 PROBLEM #2 - Too broad
});
```

- a. **ISSUE:** Stopping all events on container

11. Focus handling:

javascript

```
$(this.eInput).on('select2:open', () => {
    setTimeout(() => {
        $('.select2-search__field').focus(); // 🚩 PROBLEM #3
    }, 0);
});
```

- a. **MAJOR ISSUE:** Uses global selector \$('.select2-search__field') which targets ALL Select2 search fields on the page, causing focus to jump to other Select2 instances

3. getGui() - Returns DOM Element

javascript

```
CustomSelectEditor.prototype.getGui = function() {
    return this.eWrapper;
};
```

- **Purpose:** Returns the wrapper element for AG Grid to display
- **Status:** ✅ Correct

4. afterGuiAttached() - Post-Attachment Logic

javascript

```
CustomSelectEditor.prototype.afterGuiAttached = function() {
    setTimeout(() => {
        $(this.eInput).select2('open'); // 🚩 PROBLEM #4
    }, 50);
    this.eInput.focus();
};
```

- **ISSUES:**
 - Tries to open Select2 that may not be initialized yet
 - Focuses on raw input instead of Select2

5. `getValue()` - Returns Selected Value

javascript

```
CustomSelectEditor.prototype.getValue = function() {  
    return $(this.eInput).val();  
};
```

- **Purpose:** Returns the currently selected value
- **Status:**  Generally correct, but no error handling

6. `destroy()` - Cleanup

javascript

```
CustomSelectEditor.prototype.destroy = function() {  
    if ($(this.eInput).data('select2')) {  
        $(this.eInput).select2('destroy');  
    }  
};
```

- **Status:**  Basic cleanup, but incomplete (no event cleanup)

7. `isPopup()` - Popup Indicator

javascript

```
CustomSelectEditor.prototype.isPopup = function() {  
    return true;  
};
```

- **Purpose:** Tells AG Grid this editor should be treated as a popup
- **Status:**  Correct

8. `onKeyDown()` - Keyboard Handling

javascript

```
CustomSelectEditor.prototype.onKeyDown = function(event) {  
    const key = event.key;  
    if (key === 'Enter' || key === 'Tab') {  
        this.params.stopEditing();  
        return true;  
    }  
    if (key === 'Escape') {  
        return true;  
    }  
};
```

```
    }  
    return false;  
};
```

Method-by-Method Analysis of Your CustomTextFilter

1. Constructor

javascript

```
function CustomTextFilter() {}
```

- **Purpose:** Empty constructor for AG Grid custom filter
- **Note:** AG Grid will call `init()` to actually set up the filter

2. `init(params)` - The Main Setup Method

2A. Initial Setup

javascript

```
CustomTextFilter.prototype.init = function(params) {  
    this.params = params;  
    this.filterText = null;  
    this.filterType = 'contains'; // Default filter type  
    this.gridApi = params.api; // Store the grid API for  
    scrolling to bottom  
}
```

What it does:

- **Store parameters:** Saves AG Grid's parameters for later use
- **Initialize filter state:** `filterText = null` (no filter active initially)
- **Set default filter type:** `'contains'` (like "text contains 'abc'")
- **Store grid API:** For operations like scrolling after clearing filter

2B. Main Container Creation

javascript


```

// Create the main container with AG Grid's filter styling
this.eGui = document.createElement('div');
this.eGui.className = 'ag-filter-wrapper ag-focus-managed';
this.eGui.style.padding = '0';
this.eGui.style.backgroundColor = 'white';
this.eGui.style.borderRadius = '3px';
this.eGui.style.boxShadow = '0 1px 4px rgba(0, 0, 0, 0.2)';

```

What it creates:

- **Main wrapper div:** The container for the entire filter UI
- **AG Grid styling:** Uses AG Grid's CSS classes for consistent look
- **Visual styling:** White background, rounded corners, subtle shadow
- **this.eGui:** This becomes the main element AG Grid will display

2C. Filter Header Section (Dropdown)

```

javascript
// Create filter header with dropdown in AG Grid style
const filterHeader = document.createElement('div');
filterHeader.className = 'ag-filter-header';
filterHeader.style.padding = '6px 8px 4px 8px';
filterHeader.style.borderBottom = 'none';

```

Purpose: Creates the top section that will contain the filter type dropdown

2D. Filter Type Dropdown

```

javascript
// Create filter type dropdown styled like AG Grid's
this.eFilterType = document.createElement('select');
this.eFilterType.className = 'ag-filter-select';
// ... styling code ...

// Add focus styling (blue shadow) on click
this.eFilterType.addEventListener('focus', function() {

```

```

        this.style.boxShadow = '0 0 0 2px rgba(24, 144, 255, 0.2)';
        this.style.borderColor = '#40a9ff';
    });
    this.eFilterType.addEventListener('blur', function() {
        this.style.boxShadow = 'none';
        this.style.borderColor = '#babfc7';
    });

```

What it does:

- **Creates dropdown:** For selecting filter type (contains, equals, etc.)
- **Styling:** Matches AG Grid's visual style
- **Focus effects:** Blue glow when user clicks on it
- **Blur effects:** Removes glow when user clicks away

2E. Dropdown Options

```

javascript
// Add filter type options
const filterOptions = [
    { value: 'contains', text: 'Contém' },
    { value: 'notContains', text: 'Não contém' },
    { value: 'equals', text: 'Igual a' },
    { value: 'notEqual', text: 'Diferente de' },
    { value: 'startsWith', text: 'Começa com' },
    { value: 'endsWith', text: 'Termina com' }
];

filterOptions.forEach(option => {
    const optElement = document.createElement('option');
    optElement.value = option.value;
    optElement.text = option.text;
    this.eFilterType.appendChild(optElement);
});

```

What it creates:

- **Filter options array:** Defines all available filter types

- **Portuguese labels:** User-friendly text in Portuguese
- **Option elements:** Adds each option to the dropdown
- **Value vs Text:** value is for code, text is what user sees

2F. Search Input Section

```
javascript
// Create filter input section with search icon
const filterInputSection = document.createElement('div');
filterInputSection.className = 'ag-filter-body';
// ... styling ...

// Add search icon container
const searchIconContainer = document.createElement('div');
// ... positioning code ...

// Add a clean SVG magnifying glass icon
searchIconContainer.innerHTML = `...</svg>`;
```

What it creates:

- **Input section:** Container for the text input area
- **Search icon:** SVG magnifying glass icon positioned absolutely
- **Icon positioning:** Places icon on the left side of input field

2G. Text Input Field

```
javascript
// Create filter input with padding for the icon
this.eFilterInput = document.createElement('input');
this.eFilterInput.className = 'ag-filter-filter ag-focus-managed';
this.eFilterInput.type = 'text';
this.eFilterInput.style.width = '100%';
this.eFilterInput.style.padding = '6px 10px 6px 30px'; //
Extra padding on left for icon
```

```
// ... more styling ...
this.eFilterInput.placeholder = 'A Filtrar...';
```

Key details:

- **Text input:** Where user types their filter text
- **Left padding:** 6px 10px 6px 30px - extra 30px on left for the search icon
- **Placeholder:** "A Filtrar..." (Portuguese for "Filtering...")
- **this.eFilterInput:** Stored for later access

2H. Clear Button Section

```
javascript
// Create clear button section with AG Grid styling
const filterFooter = document.createElement('div');
// ... styling ...

// Create clear button
this.eClearButton = document.createElement('button');
this.eClearButton.innerHTML = 'Limpar Filtro';
// ... styling and hover effects ...
```

What it creates:

- **Footer section:** Bottom area of the filter
- **Clear button:** Button to remove the current filter
- **Hover effects:** Changes background color on mouse over
- **Portuguese text:** "Limpar Filtro" (Clear Filter)

2I. Event Listeners

```
javascript
// Add event listeners
this.eFilterType.addEventListener('change',
this.onFilterTypeChanged.bind(this));
this.eFilterInput.addEventListener('input',
this.onFilterChanged.bind(this));
```

```
this.eClearButton.addEventListener('click',  
this.onClearFilter.bind(this));
```

What it does:

- **Dropdown change:** When user selects different filter type
- **Input typing:** When user types in the text field (real-time filtering)
- **Clear button:** When user clicks "Clear Filter"
- **.bind(this):** Ensures this refers to the filter instance inside the event handlers

2J. State Restoration

```
javascript  
// Initialize with saved state if available  
const existingModel = params.filterModel;  
if (existingModel) {  
    this.filterType = existingModel.type || 'contains';  
    this.filterText = existingModel.filter || '';  
    this.eFilterType.value = this.filterType;  
    this.eFilterInput.value = this.filterText;  
}
```

Purpose: If the filter was previously applied, restore its settings when reopened

3. afterGuiAttached() – Post-Display Setup

```
javascript  
CustomTextFilter.prototype.afterGuiAttached = function() {  
    // Auto-focus the filter textbox when the filter is  
    attached  
    setTimeout(() => {  
        this.eFilterInput.focus();  
        this.eFilterInput.select(); // Select any existing  
        text  
    })  
}
```

```
    }, 0);  
};
```

What it does:

- **Auto-focus:** Automatically puts cursor in the text input
- **Text selection:** If there's existing text, selects it all
- **setTimeout:** Ensures DOM is ready before focusing

4. `getGui()` - Returns Filter UI

```
javascript  
CustomTextFilter.prototype.getGui = function() {  
    return this.eGui;  
};
```

Purpose: AG Grid calls this to get the filter's HTML element to display

5. `isFilterActive()` - Filter State Check

```
javascript  
CustomTextFilter.prototype.isFilterActive = function() {  
    return this.filterText !== null && this.filterText !== '';  
};
```

What it checks: Whether there's currently an active filter (user has typed something)

6. `doesFilterPass(params)` - The Core Filtering Logic (Updated)

```
Javascript  
CustomTextFilter.prototype.doesFilterPass = function(params) {  
    // Get cell value  
    const cellValue = this.params.valueGetter(params);
```

```

const filterText = this.filterText;

// Early exit for null/undefined
if (cellValue === null || cellValue === undefined) {
    return false;
}

// Get column field to determine data type
const columnField = this.params.column.colDef.field;

// Choose the right Dictionary for array columns
let dictionary;
if (columnField === 'servicosIds') {
    dictionary = this.params.context.servicesDict;
} else if (columnField === 'viaturasIds') {
    dictionary = this.params.context.viaturasDict;
}

// Convert to searchable string (handles both arrays and
single values)
let cellValueStr;
if (Array.isArray(cellValue)) {
    // Handle array columns (servicosIds, viaturasIds)
    const readableValues = cellValue.map(id => {
        const dictValue = dictionary[id];
        return dictValue?.Nome || dictValue || id; //
Handle both object and string values
    });
    cellValueStr = readableValues.join(' ').toLowerCase();
} else {
    // Handle single value columns (motoristaId, data,
etc.)
    cellValueStr = cellValue.toString().toLowerCase();
}

// Prepare filter text for comparison
const filterTextLower = filterText.toLowerCase();

```

```

// Apply filter based on filter type
switch (this.filterType) {
    case 'equals':
        return cellValueStr === filterTextLower;
    case 'notEqual':
        return cellValueStr !== filterTextLower;
    case 'contains':
        return cellValueStr.indexOf(filterTextLower) >= 0;
    case 'notContains':
        return cellValueStr.indexOf(filterTextLower) === -
1;

    case 'startsWith':
        return cellValueStr.indexOf(filterTextLower) ===
0;

    case 'endsWith':
        return cellValueStr.indexOf(filterTextLower,
cellValueStr.length - filterTextLower.length) !== -1;
    default:
        return false;
}
};

```

What it does:

- **Gets cell value:** Extracts the value from the current row/cell
- **Null check:** Returns false for empty values
- **Column detection:** Identifies which column is being filtered to choose correct dictionary
- **Dictionary selection:** Uses servicesDict for servicosIds, viaturasDict for viaturasIds
- **Array handling:** For array columns, converts IDs to readable values using dictionary lookup
 - **Object values:** Extracts .Nome property (for services)
 - **String values:** Uses the string directly (for viaturas)
 - **Fallback:** Uses original ID if dictionary lookup fails
- **Array joining:** Combines multiple values into single searchable string with spaces
- **Single value handling:** Converts non-array values directly to lowercase string

- **Case conversion:** Converts both cell and filter text to lowercase for case-insensitive comparison
- **Filter logic:** Uses switch statement to apply different filter types:
 - **equals:** Exact match
 - **contains:** Cell value contains the filter text (works across multiple array values)
 - **startsWith:** Cell value begins with filter text
 - **endsWith:** Cell value ends with filter text
 - **notContains/notEqual:** Opposite logic

Key Enhancement: Now supports filtering by **human-readable values** instead of raw IDs for array columns, while maintaining backward compatibility with single-value columns.

7. getModel() - Save Filter State

javascript

```
CustomTextFilter.prototype.getModel = function() {
    if (!this.isFilterActive()) {
        return null;
    }

    return {
        type: this.filterType,
        filter: this.filterText
    };
};
```

Purpose: Returns current filter settings so AG Grid can save/restore them

8. setModel(model) - Restore Filter State

javascript

```
CustomTextFilter.prototype.setModel = function(model) {
    if (model) {
        this.filterType = model.type;
    }
};
```

```

        this.filterText = model.filter;
        this.eFilterType.value = this.filterType;
        this.eFilterInput.value = this.filterText;
    } else {
        this.filterType = 'contains';
        this.filterText = null;
        this.eFilterType.value = this.filterType;
        this.eFilterInput.value = '';
    }
};

```

What it does: Restores filter to a previously saved state (or clears it)

9. Event Handlers

9A. onFilterTypeChanged()

```

javascript
CustomTextFilter.prototype.onFilterTypeChanged = function() {
    this.filterType = this.eFilterType.value;
    this.params.filterChangedCallback();
};

```

When called: User changes dropdown selection **What it does:** Updates filter type and triggers grid refresh

9B. onFilterChanged()

```

javascript
CustomTextFilter.prototype.onFilterChanged = function() {
    const inputValue = this.eFilterInput.value;
    if (!inputValue || inputValue === '') {
        this.onClearFilter();
        return;
    }

    this.filterText = inputValue;
};

```

```
        this.params.filterChangedCallback();
    };
```

When called: User types in text input **What it does:** Updates filter text and triggers grid refresh (or clears if empty)

9C. *onClearFilter()*

javascript

```
CustomTextFilter.prototype.onClearFilter = function() {
    this.filterText = null;
    this.eFilterInput.value = '';
    this.params.filterChangedCallback();

    // Scroll to the bottom row when filter is cleared
    setTimeout(() => {
        const gridApi = this.params.api;
        const rowCount = gridApi.getDisplayedRowCount();
        if (rowCount > 0) {
            const lastRowIndex = rowCount - 1;
            gridApi.ensureIndexVisible(lastRowIndex,
'bottom');
        }
    }, 100);
};
```

What it does:

- Clears the filter
- Empties the input field
- Refreshes grid
- **Bonus feature:** Scrolls to bottom of grid after clearing

10. *getModelAsString()* - Display Current Filter

javascript

```
CustomTextFilter.prototype.getModelAsString = function() {
    if (!this.isFilterActive()) {
        return '';
    }
};
```

```

    }

    const filterType = this.filterType;
    let filterTypeDisplay = '';

    switch (filterType) {
        case 'equals': filterTypeDisplay = 'igual a'; break;
        case 'notEqual': filterTypeDisplay = 'diferente de';
break;
        case 'contains': filterTypeDisplay = 'contém'; break;
        case 'notContains': filterTypeDisplay = 'não contém';
break;
        case 'startsWith': filterTypeDisplay = 'começa com';
break;
        case 'endsWith': filterTypeDisplay = 'termina com';
break;
    }

    return `${filterTypeDisplay} "${this.filterText}"`;
};

```

Purpose: Creates human-readable description of current filter (e.g., "contém 'João'")

Overall Architecture:

This is a **comprehensive custom filter** that:

1. **Looks like AG Grid:** Uses AG Grid's styling classes
2. **Multiple filter types:** Contains, equals, starts with, etc.
3. **Portuguese localized:** All text in Portuguese
4. **Auto-focus:** Puts cursor in input when opened
5. **Visual feedback:** Hover effects, focus styling
6. **State management:** Can save/restore filter settings
7. **Smart clearing:** Scrolls to bottom when filter is cleared

It's essentially a complete replacement for AG Grid's default text filter with enhanced Portuguese localization and custom styling!